



Tickema

OBJECT DESIGN DOCUMENT

Corso Triennale Informatica – Università di Salerno
Corso di Ingegneria del Software – Prof C. Gravino
Autori (Gruppo NC03): Di Tella Sara, Iuorio Irene,
Maurelli Raffaella, Pascarella Laura

Revision History

Data	Versione	Descrizione	Autori
25/11/2025	0.1	Elaborazione sezione dedicata alle linee guida e organizzazione in package	SDT
25/11/2025	0.2	Elaborazione sezione dedicata alla specifica delle interfacce	II
27/11/2025	0.3	Elaborazione sezione relativa a precondizioni, post-condizioni e invarianti	SDT
27/11/2025	0.4	Elaborazione sezione dedicata al glossario	SDT
28/11/2025	0.5	Rielaborazione e modifiche relative alla sezione di specifica delle interfacce e alla sezione relativa ai contratti	II, LP, SDT
28/11/2025	0.6	Definizione e scelta dei design pattern	II, SDT
29/11/2025	0.7	Elaborazione sezione dedicata ai design patterns	II
07/12/2025	1.0	Stesura completa e inserimento argomentazioni sulle scelte fatte al documento	SDT
11/12/2025	2.0	Revisione generale del documento	Tutto il team
16/12/2025	3.0	Ulteriore Revisione generale del documento e definizione estetica delle varie sezioni	II
03/01/2026	4.0	Modifica Specifica delle interfacce con aggiunta metodi mancanti. Modifica Facede Pattern	II

Team Members

Nome	Ruolo nel progetto	Acronimo	Informazioni di contatto
Irene Iuorio	Team Member	II	i.iuorio2@studenti.unisa.it
Laura Pascarella	Team Member	LP	l.pascarella5@studenti.unisa.it
Raffaella Maurelli	Team Member	RM	r.maurelli@studenti.unisa.it
Sara Di Tella	Team Member	SDT	s.ditella@studenti.unisa.it

Glossario

Di seguito sono elencati i termini tecnici utilizzati nel documento, per evitare ambiguità.

Termine	Interfaccia
Bean (o POJO)	<i>Plain Old Java Object</i> . Oggetto semplice utilizzato per trasportare dati tra i livelli architetturali. Corrisponde alle entità del database.
DAO	<i>Data Access Object</i> . Design pattern strutturale che astrae e incapsula tutti gli accessi alla fonte dati (Database). Il DAO gestisce la connessione e le query SQL.
Facade	Design pattern strutturale che fornisce un'interfaccia unificata e semplificata a un insieme di interfacce in un sottosistema. In Tickema, i "Service" fungono da Facade per la logica di business.
Strategy	Pattern comportamentale che definisce una famiglia di algoritmi (strategie), li incapsula e li rende intercambiabili. In Tickema, è utilizzato per gestire i diversi metodi di pagamento (Carta, Saldo, Misto) in modo flessibile.
Design by Contract	Metodologia di progettazione software che definisce obblighi formali (<i>Pre-condizioni</i> , <i>Post-condizioni</i> , <i>Invarianti</i>) per le interfacce delle classi.

UpperCamelCase	Convenzione di nomenclatura dove ogni parola inizia con la maiuscola (es. GestioneAcquisti). Usata per le Classi.
LowerCamelCase	Convenzione dove la prima parola è minuscola e le successive maiuscole (es. calcolaTotale). Usata per metodi e variabili.
Service	Package che costituisce il Logic Tier (Livello di Business). Contiene le classi che applicano le regole di business e l'orchestrazione, implementando il pattern Facade.
Repository	Package che costituisce il Data Access Layer. Implementa il sottosistema SST (Storage) e contiene le classi DAO, gestendo l'accesso diretto al database.
Entity	Package che rappresenta il sottosistema SDB (Persistenza) in termini di oggetti codice. Contiene i Bean (POJO) che mappano le tabelle del database.
Exception	Package dedicato alla gestione degli errori. Contiene le eccezioni personalizzate utilizzate per segnalare errori di business o di sistema tra i diversi livelli.

Sommario

Revision History	2
Team Members	3
Glossario	3
1. Introduzione	6
1.1 Scopo del Sistema.....	6
2.Abbreviazioni e Acronimi	6
3. Riferimenti	7
4.Linee guida.....	7
4.1 Convenzioni	7
5.Struttura del sistema	8
5.1 Packages	8
5.1.1 Package Control.....	9
5.1.2 Package Service	9
5.1.3 Package Repository.....	10
5.1.4 Package Entity.....	10
5.1.5 Package Exception	10
6.Specifica delle interfacce	11
7.1 Invarianti delle classi	20
8. Design patterns	22
8.1 Facade Pattern.....	22
8.2 Strategy pattern	24

1. Introduzione

1.1 Scopo del Sistema

Lo scopo primario del sistema è di digitalizzare e ottimizzare il processo di acquisto di biglietti cinematografici e la gestione della programmazione per una struttura multisala, oltre ad un sistema di validazione dei biglietti per il personale addetto del cinema.

2. Abbreviazioni e Acronimi

A seguire sono riportate le abbreviazioni che è possibile incontrare durante la lettura di tale documento e dei documenti successivi.

Abbreviazione	Descrizione
API	Interfacce per la comunicazione tra moduli software
SGA	Sottosistema Gestore Acquisti
SGC	Sottosistema Gestore Catalogo
SGP	Sottosistema Gestore Programmazioni
SGU	Sottosistema Gestore Utenti
SST	Sottosistema Storage
SDB	Sottosistema Persistenza

NOTA: Le definizioni complete di tutti i termini tecnici e degli attori sono consultabili nel Glossario e nei Riferimenti dei documenti precedenti.

3. Riferimenti

Di seguito sono riportati i riferimenti a risorse che supportano l'analisi e la progettazione del sistema:

Nome Riferimento	Riferimento
Testo di Riferimento	“Object-Oriented Software Engineering: Conquering Complex and Changing Systems”, terza edizione, Bernd Bruegge & Allen Dutoit, 2014.
Documento RAD	IS_NC03_RAD
Documento SDD	IS_NC03_SDD

4.Linee guida

4.1 Convenzioni

Di seguito sono riportate una serie di linee guida utili in fase implementativa. Queste permettono una maggiore leggibilità delle componenti del sistema, garantendo una revisione più veloce ed organizzata, assicurando manutenibilità e coerenza nel codice dei diversi membri del team.

Regola	Descrizione
Nomi delle classi e interfacce	I nomi delle classi e delle interfacce devono essere sostantivi al singolare e dovranno rispettare la notazione UpperCamelCase. Per identificare il ruolo architetturale, è obbligatorio l'uso dei suffissi: se una classe è un Data Access Object il nome dovrà terminare con "DAO"; se è una Business Logic Facade dovrà terminare con "Service"; se è un Controller HTTP dovrà terminare con "Servlet"; se è un'eccezione dovrà terminare con "Exception". Le Entity/Bean non richiedono suffissi
Nomi dei metodi	I nomi dei metodi devono essere verbi o frasi verbali che descrivono l'azione eseguita e dovranno rispettare la notazione lowerCamelCase. I nomi delle variabili e dei parametri devono essere sostantivi brevi ma descrittivi e dovranno seguire la notazione lowerCamelCase. Non sono ammessi nomi generici tranne che per i contatori di loop.

Nomi delle variabili e costanti	Le costanti dovranno seguire la notazione UPPER_SNAKE_CASE
Gestione degli errori	I metodi che possono generare eccezioni controllate devono dichiararle esplicitamente nella loro firma tramite la clausola throws

5.Struttura del sistema

5.1 Packages

Questa sezione mostra l'organizzazione logica del codice sorgente. Tale schematizzazione si basa sull'architettura a livelli e sulla scomposizione modulare, definite nel System Design Document (SDD).

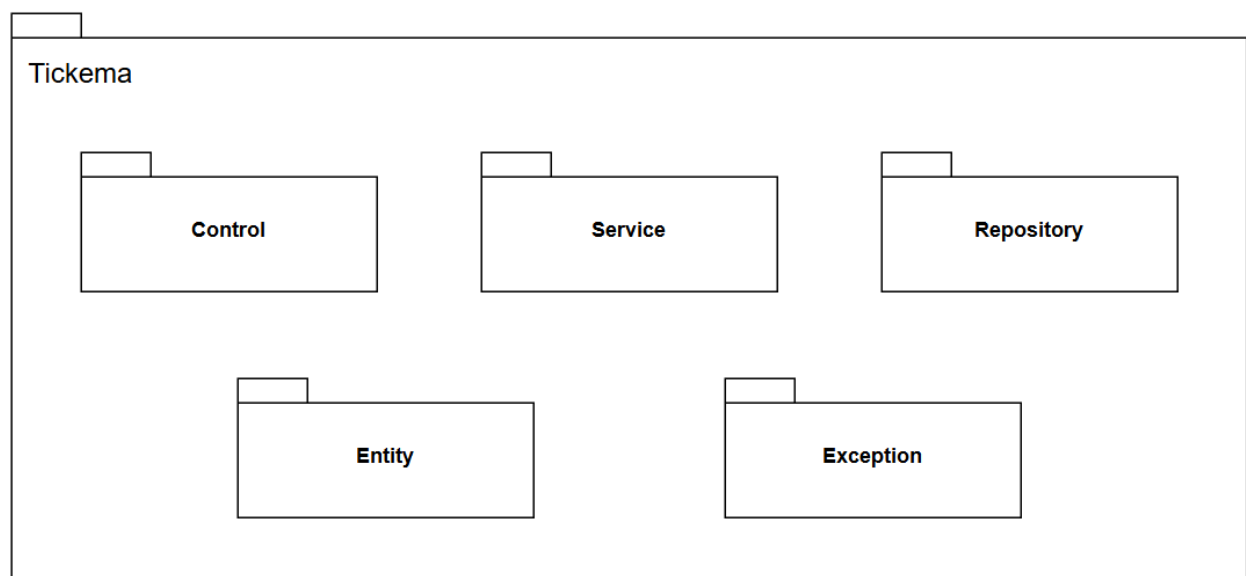
Questa struttura costituisce una prima ipotesi progettuale, pensata per fornire una visione iniziale dell'organizzazione del codice.

L'impostazione è indicativa e non definitiva, e potrà evolvere nel corso dello sviluppo.

Al livello più alto, sono stati definiti i package corrispondenti ai layer architetturali principali:

Tickema

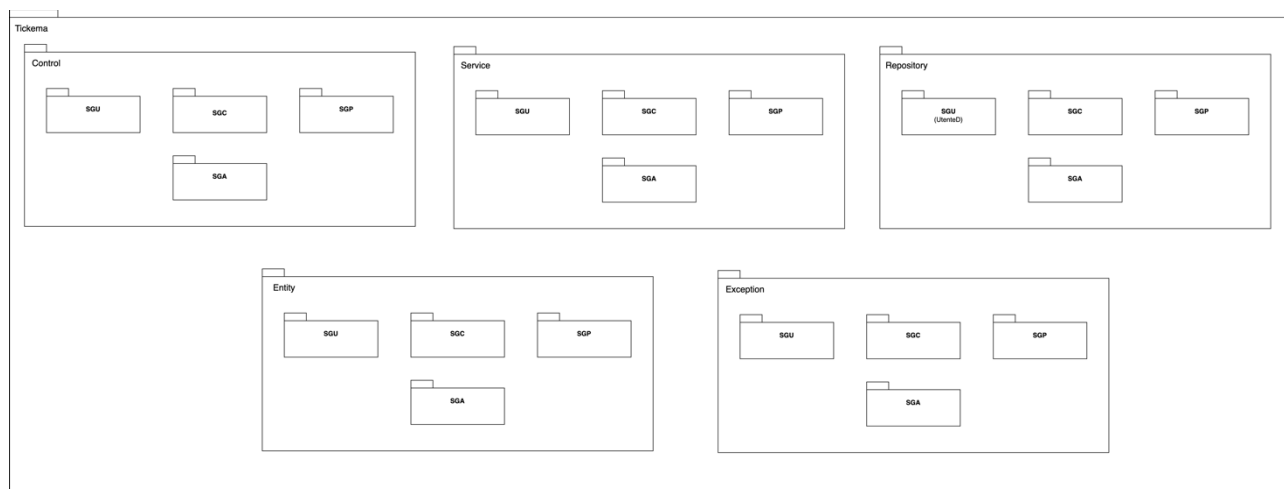
- **Control** (Gestione delle richieste HTTP/Servlet)
- **Service** (Logica di Business e Facade)
- **Repository** (Implementazione del sottosistema SST - Storage per l'accesso ai dati)
- **Entity** (Rappresentazione del sottosistema SDB - Persistenza tramite oggetti Java)
- **Exception** (Gestione degli errori)



È da notare che ai package rappresentanti i vari livelli architetturali sono stati aggiunti altri due package, necessari a gestire le classi delineate come Entity e le eccezioni che possono essere sollevate da una certa funzionalità di sistema.

All'interno di ciascuno di questi package, la struttura viene ancora una volta suddivisa usando i sottosistemi funzionali (SGU, SGC, SGP, SGA) per mantenere l'ordine logico e la separazione delle responsabilità.

L'organizzazione del codice sorgente segue un pattern architetturale preciso per garantire la massima coerenza e manutenibilità. Si noti dunque la scomposizione in sottosistemi funzionali (SGU, SGC, SGP, SGA) che viene replicata in modo speculare all'interno di ogni layer.



Lo schema è consultabile in alta risoluzione scaricando il relativo file PDF dalla cartella; la visualizzazione diretta tramite anteprima Drive potrebbe comprometterne la nitidezza.

Di seguito viene descritta la responsabilità specifica di ciascun package architetturale rappresentato nel diagramma:

5.1.1 Package Control

Contiene le classi Servlet che gestiscono le richieste HTTP e l'interazione con l'utente. Ogni sotto-package funzionale isola le Servlet relative a quella specifica area

- **SGU** (Contiene: AdminAcquistiServlet, AdminDashboardServlet, AdminUtenteServlet, StaffDashboardServlet, UtenteServlet)
- **SGC** (Contiene: CatalogoServlet, DettaglioFilmServlet, DettaglioProgrammazioniServlet, FilmServlet, HomeServlet, InfoPagesServlet, ProgrammazioniServlet)
- **SGP** (Contiene: ProgrammazioneServlet)
- **SGA** (Contiene: AcquistoServlet, AnnullaCheckoutServlet, BigliettoServlet, ValidazioneServlet)

5.1.2 Package Service

Questo package contiene le interfacce e le classi che incapsulano la logica applicativa, la gestione delle transazioni e il coordinamento delle operazioni complesse, agendo come *Facade* per il layer sottostante.

- **SGU** (Contiene: AutenticazioneService, UtenteService)
- **SGC** (Contiene: FilmService)
- **SGP** (Contiene: PostoService, ProgrammazioneService, RisultatoAssegnazione, SalaService, SlotOrariService, TariffaService)
- **SGA** (Contiene: AcquistoFacade, AcquistoService, BigliettoService, PagamentoService, RisultatoAcquisto, SaldoService)

5.1.3 Package Repository

Questo package costituisce l'interfaccia verso il database. Implementa concretamente il sottosistema SST (Storage) attraverso il pattern DAO. Le classi qui contenute si occupano esclusivamente di eseguire query SQL e mappare i risultati, senza applicare logica di business.

- **SGU** (Contiene: UtenteDAO)
- **SGC** (Contiene: FilmDAO)
- **SGP** (Contiene: ProgrammazioneDAO, SalaDAO, TariffaDAO, PostoDAO, SlotOrariDAO)
- **SGA** (Contiene: AcquistoDAO, BigliettoDAO, PagamentoDAO)

5.1.4 Package Entity

Questo package rappresenta il sottosistema SDB (Persistenza) in termini di oggetti Java. Contiene i Bean (POJO) che mappano le tabelle del database relazionale. Queste classi attraversano tutti i layer dell'architettura 3-Tier fungendo da *Data Transfer Objects*.

- **SGU** (Contiene: Utente)
- **SGC** (Contiene: Film)
- **SGP** (Contiene: Programmazione, Sala, Posto, SlotOrari, Tariffa)
- **SGA** (Contiene: Acquisto, Biglietto, Pagamento)

5.1.5 Package Exception

Questo package centralizza la gestione degli errori. Contiene eccezioni personalizzate (Checked e Unchecked) suddivise per sottosistema, permettendo al Controller di distinguere tra errori di business ed errori di sistema.

- **SGU** (Contiene: CredenzialiNonValideException, EmailGiaRegistrataException, PasswordDiverseException)
- **SGC**
- **SGP** (AssegnazionePostiException, ConteggioPostiException, GenerazionePostiException, LiberazionePostiException, NessunPostoDisponibileException, NumeroPostiDaAssegnareException, CalcoloPrezzoTariffatoException, CreazioneProgrammazioneException, CreazioneProgrammazioneMultiplaException, DurataFilmException, EliminazioneProgrammazioneException, ProgrammazioneNonTrovataException,

CalcoloSaleDisponibiliException, ModificaSalaException, NomeSalaException, SalaNonTrovataException, CalcoloSlotDisponibiliException, CreazioneSlotException, EliminazioneSlotException, ModificaSlotException, SlotNonTrovatoException, CalcoloPrezzoScontatoException, CreazioneTariffaException, ModificaTariffaException, PercentualeScontoNonValidException, RecuperoTariffeException, TariffaNonTrovataException)

- **SGA** (Contiene: AcquistoNonTrovatoException, AcquistoNonValidoException, RimborsoNonConsentitoException, SalvataggioAcquistoException, PagamentoNonTrovatoException, PagamentoNonValidoException, SalvataggioPagamentoException, AggiornamentoSaldoException, OperazioneSaldoNonValidaException, SaldoInsufficiente, TipoAccountNonValidoException, UtenteNonTrovatoException, AggiornamentoBigliettoException, DataValidazioneNonValidaException, StatoBigliettoNonValidoException, ValidazioneNonRiuscitaException)

6. Specifica delle interfacce

Nell'ODD la specifica delle interfacce è stata concentrata sul **Service Layer**, in quanto questo livello rappresenta il principale punto di coordinamento della logica applicativa e delle funzionalità offerte dal sistema.

Tale scelta è stata adottata per ottimizzare tempi e risorse di progetto, tenendo conto dei vincoli di budget, privilegiando la definizione delle interfacce più rilevanti dal punto di vista funzionale e rimandando il dettaglio degli altri livelli alle fasi successive di sviluppo.

Nome: AutenticazioneService
+ registraUtente(String nome, String cognome, String email, String password, String confermaPassword, String numeroDiTelefono): Utente + login(String email, String passwordInserita): Utente + logout(int idAccount): boolean + verificaAutenticazione(String token): boolean + modificaProfilo(int idAccount, String nome, String cognome, String numeroDiTelefono): boolean + modificaCredenziali(int idAccount, String email, String password, String confermaPassword): boolean

Nome: FilmService
+ visualizzaDettagliFilm(int idFilm): Film + visualizzaTuttiFilm(): List<Film> + cercaFilmPerTitolo(String titolo): List<Film> + getFilmPerGenere(String genere): List<Film> + getFilmPerAnno(int anno): List<Film> + getFilmPerRegista(String regista): List<Film> + creaFilm(String trama, String titolo, int anno, String regista, String genere, int durata, String locandina): Film + modificaFilm(int idFilm, String trama, String titolo, int anno, String regista, String genere, int durata, String locandina): boolean

- + eliminaFilm(int idFilm): boolean
- + filmEsiste(int idFilm): boolean
- + contaFilm(): int
- + getGeneriDisponibili(): List<String>
- + ricercaFilmConFiltri(String titolo, String genere, Integer anno, Integer durataMin, Integer durataMax, LocalDate dataProiezione, boolean soloInProgrammazione): List<Film>
- + getAllGeneri(): List<String>
- + getAllAnni(): List<Integer>
- + searchFilmsByTitolo(String query): List<Film>
- + getFilmsByGenere(String genere): List<Film>

Nome: ProgrammazioneService

- + visualizzaProgrammazioniFilm(int idFilm): List<Programmazione>
- + getProgrammazioniPerData(LocalDate data): List<Programmazione>
- + getProgrammazioniPerSala(int idSala, LocalDate data): List<Programmazione>
- + getAllProgrammazioni(): List<Programmazione>
- + creaProgrammazioneSingola(LocalDate date, String tipo, double prezzoBase, LocalTime orainizio, int idFilm, int idSala, int idSlotOrario, Integer idTariffa): Programmazione
- + creaProgrammazioneMultipla(List<LocalDate> date, String tipo, double prezzoBase, List<LocalTime> orainizio, int idFilm, List<Integer> idSala, List<Integer> idSlotOrario, List<Integer> idTariffa): List<Programmazione>
- + modificaProgrammazione(int idProgrammazione, LocalDate date, String tipo, double prezzoBase, String stato, LocalTime orainizio, int idFilm, int idSala, int idSlotOrario, Integer idTariffa): boolean
- + eliminaProgrammazione(int idProgrammazione): boolean
- + getProgrammazioneById(int id): Programmazione
- + getProgrammazioneByKey(int id): Programmazione
- creaProgrammazioneSingolaInternal(LocalDate date, String tipo, double prezzoBase, LocalTime orainizio, int idFilm, int idSala, int idSlotOrario, Integer idTariffa): Programmazione
- processaRimborsiAutomatici(int idProgrammazione): void
- calcolaPrezzoTariffato(double prezzoBase, int idTariffa): double
- rollback(): void
- caricaRelazioniProgrammazione(Programmazione prog): void

Nome: SlotOrarioService

- + creaSlotOrario(LocalTime orainizio, LocalTime oraFine, String stato, LocalDate data): SlotOrari
- + modificaSlotOrario(int idSlot, LocalTime orainizio, LocalTime oraFine, String stato, LocalDate data): boolean
- + eliminaSlotOrario(int idSlot): boolean
- + visualizzaSlotDisponibili(int idSala, LocalDate data): List<SlotOrari>

+ getSlotOrarioById(int idSlot): SlotOrari

Nome: AcquistoService

+ creaAcquisto(Utente utente, int numeroBiglietti, double importoTotale): Acquisto
+ rimborsaAcquisto(int idAcquisto): boolean
+ getAcquistoById(int idAcquisto): Acquisto
+ getAcquistiUtente(int idAccount): List<Acquisto>
+ getAcquistiPerStato(String stato): List<Acquisto>
+ getAcquistiPerPeriodo(LocalDateTime dataInizio, LocalDateTime dataFine): List<Acquisto>
+ contaAcquistiUtente(int idAccount): int
+ getAcquistiCompletatiUtente(int idAccount): List<Acquisto>
+ getAcquistiRimborsatiUtente(int idAccount): List<Acquisto>
+ isAcquistoRimborsabile(int idAcquisto): boolean
+ getAllAcquisti(): List<Acquisto>
+ contaTotaleAcquisti(): int
+ contaAcquistiPerStato(String stato): int
+ calcolaImportoTotaleAcquisti(): double
- validaNumeroBiglietti(int numeroBiglietti): void
- validaImporto(double importo): void
- validaStato(String stato): void

Nome: PagamentoService

+ effettuaPagamento(int idAcquisto, String metodoPagamento, double importo, String tipo): Pagamento
+ getPagamentiPerAcquisto(int idAcquisto): List<Pagamento>
+ verificaPagamento(int idPagamento): boolean
+ calcolaTotalePagato(int idAcquisto): double

Nome: BigliettoService

+ generaBiglietto(int idAcquisto, int idProgrammazione, int idPosto, double prezzoFinale): Biglietto
+ getBigliettoById(int idBiglietto): Biglietto
+ getBigliettoByQRCode(String qrCode): Biglietto
+ getBigliettiPerAcquisto(int idAcquisto): List<Biglietto>
+ getBigliettiPerUtente(int idAccount, boolean soloValidi): List<Biglietto>
+ rimborsaBiglietto(int idBiglietto): boolean
+ scadiBiglietto(int idBiglietto): boolean
+ scadiBigliettiProgrammazione(int idProgrammazione): int
+ isBigliettoUtilizzabile(int idBiglietto): boolean

+ getBigliettiPerProgrammazione(int idProgrammazione): List<Biglietto> -
generaQRCodeUnivoco(): String
- validaStato(String stato): void
- caricaRelazioniBiglietto(Biglietto biglietto): void

Nome: ValidazioneService

+ validaBiglietto(String qrCode, int idPersonale): boolean
+ verificaStatoBiglietto(String qrCode): StatoBiglietto
+ getBigliettiValidati(int idPersonale, LocalDate data): List<Biglietto>
+ contaValidazioniOggi(int idPersonale): int
- generaMessaggioErroreStato(Biglietto biglietto): String

Nome: PostoService

+ getPostiDisponibili(int idProgrammazione): List<Posto>
+ contaPostiDisponibili(int idProgrammazione): int
+ getPostiPerSala(int idSala, int idProgrammazione): List<Posto>
+ verificaDisponibilitaPosti(int idProgrammazione, int numeroBiglietti): List<Posto>
+ verificaDisponibilitaPosto(int idPosto, int idProgrammazione): boolean
+ assegnaPostiAutomatico(List<Posto> postiDisponibili, int numeroPostiDaAssegnare): RisultatoAssegnazione
+ assegnaPosti(List<Posto> posti, int idAcquisto): void
+ liberaPosti(List<Integer> idPosti, int idProgrammazione): void
+ generaPostiPerProgrammazione(int idProgrammazione): void
+ occupaTemporaneamente(List<Posto> posti): boolean
+ liberaPostiDaCheckout(List<Posto> posti): void
- cercaGruppoAdiacente(List<Posto> posti, int dimensione): List<Posto>
- cercaGruppiParziali(List<Posto> posti, int totale): List<Posto>
- calcolaPercentualeAdiacenza(List<Posto> posti): double
- formatListaPosti(List<Posto> posti): String
- rollback(): void

Nome: Sala Service

+ creaSala(String nome, int numeroDiFile, int capienza, int numeroPostiPerFila): Sala
+ modificaSala(int idSala, String nome, int numeroDiFile, int capienza, int numeroPostiPerFila): boolean
+ visualizzaDettagliSala(int idSala): Sala
+ visualizzaTutteLeSale(): List<Sala>
+ getSaleDisponibili(LocalDate data, LocalTime oraInizio, int durataMinuti): List<Sala>
- rollback(): void

Nome: TariffaService
+ creaTariffa(String tipo, String nome, int percentualeSconto): Tariffa + modificaTariffa(int idTariffa, String tipo, String nome, int percentualeSconto): boolean + visualizzaTariffe(): List<Tariffa> + getTariffaById(int idTariffa): Tariffa + applicaSconto(double prezzoBase, int idTariffa): double

Nome: SaldoService
+ getSaldoDisponibile(int idAccount): double + utilizzaSaldo(int idAccount, double importo): boolean + rimborsaSaldo(int idAccount, double importo): boolean + isSaldoSufficiente(int idAccount, double importoRichiesto): boolean + calcolaImportoSaldoDaUsare(int idAccount, double importoTotale): double + calcolaDifferenzaConCarta(int idAccount, double importoTotale): double

Nome: UtenteService
+ getAllUtenti(): List<Utente> + getUtentiPerTipo(String tipoAccount): List<Utente> + cercaUtenti(String query): List<Utente> + contaTotaleUtenti(): int + contaUtentiPerTipo(String tipoAccount): int + getUtenteById(int idAccount): Utente

7. Precondizioni e postcondizioni

Qui di seguito vengono riportate le precondizioni e postcondizioni di ogni metodo definito nella sezione precedente. A questi saranno associati:

- un nome, ripreso dalla firma definite nell'interfaccia d'appartenenza,
- il nome dell'interfaccia a cui il metodo appartiene,
- una descrizione necessaria a comprendere la funzionalità rappresentata dal metodo stesso,
- una o più precondizioni,
- e una o più postcondizioni.

Nome	Interfaccia	Descrizione
registraUtente	AutenticazioneServie	Registra un nuovo account utente nel sistema verificando l'unicità dell'e-mail e la sicurezza della password.
Precondizioni		
Non deve esistere già un utente registrato con la stessa e-mail nel sistema. La password fornita deve rispettare i criteri di sicurezza minimi: deve essere composta da almeno 6 caratteri con aggiunta di almeno un carattere speciale e di una lettera maiuscola. I campi obbligatori non devono essere vuoti.		
Postcondizioni		
Viene creato un nuovo oggetto Account nel sistema con i dati forniti. Il saldo iniziale dell'utente è impostato a zero. L'utente risulta abilitato ad accedere.		

Nome	Interfaccia	Descrizione
creaSala	SalaService	Crea una nuova sala cinematografica e genera automaticamente i posti a sedere corrispondenti.
Precondizioni		
Il numero di file e il numero di posti per fila devono essere maggiori di zero. Non deve esistere un'altra sala con lo stesso nome nel cinema. La capienza deve essere coerente al numeroDiFile * numeroPostiPerFila.		
Postcondizioni		
Viene creata la nuova entità Sala. Vengono generati automaticamente tutti i singoli oggetti Posto associati alla sala. Tutti i posti generati sono inizializzati nello stato *DISPONIBILE*.		

Nome	Interfaccia	Descrizione
creaProgrammazione	ProgrammazioneService	Pianifica la proiezione di un film in una specifica sala, data e ora.
Precondizioni		
Il film specificato deve esistere nel catalogo. La sala specificata deve esistere. La data di inizio deve essere futura rispetto al momento della creazione. Lo SlotOrario indicato deve essere libero per quella sala in quella data.		
Postcondizioni		
Viene creata una nuova istanza di Programmazione. Lo stato della programmazione è impostato su *DISPONIBILE*. Lo SlotOrario corrispondente viene settato a *OCCUPATO*.		

Nome	Interfaccia	Descrizione
eliminaProgrammazione	ProgrammazioneService	Annulla o rimuove una programmazione esistente.
Precondizioni		
La programmazione deve esistere.		
Postcondizioni		
Lo stato della programmazione viene impostato su *ANNULLATA*. La sala torna disponibile per quello slot orario preso in considerazione. Lo stato dello slot orario viene settato da *OCCUPATO* a *DISPONIBILE*. Tutti gli eventuali biglietti venduti per questa programmazione devono essere stati tutti rimborsati.		

Nome	Interfaccia	Descrizione
creaAcquisto	AcquistoService	Gestisce la transazione per l'acquisto di biglietti, verificando disponibilità e posti.
Precondizioni		
L'account utente deve esistere. La programmazione deve essere attiva per essere scelta e acquistata. Il numero di biglietti richiesti deve essere maggiore di zero. Deve esserci un numero di posti disponibili nella sala maggiore o uguale al numero di biglietti richiesti.		
Postcondizioni		
Viene creato un Acquisto collegato all'utente. L'importo totale è calcolato correttamente. I posti selezionati vengono marcati come *OCCUPATO* per quella programmazione presa in considerazione.		

Nome	Interfaccia	Descrizione
generaBiglietto	BigliettoService	Genera il titolo digitale a seguito di un acquisto che è stato confermato.
Precondizioni		
L'acquisto deve essere stato confermato.		
Postcondizioni		
Il biglietto viene creato con stato *EMESSO*. Viene generato un codice QR univoco associato al biglietto. Il biglietto è associato senza equivoci a un posto, e una programmazione specifici.		

Nome	Interfaccia	Descrizione
rimborsoAcquisto	AcquistoService	Annulla un acquisto e rimborsa l'utente, liberando i posti occupati
Precondizioni		
L'acquisto deve esistere nel sistema. Nessuno dei biglietti associati all'acquisto deve essere stato già validato. Lo stato della programmazione deve essere *DISPONIBILE*.		
Postcondizioni		
Lo stato dell'acquisto diventa "RIMBORSATO". Tutti i biglietti associati vengono invalidati. I posti presi attraverso i biglietti tornano ad essere disponibili per quella programmazione. L'importo speso viene riaccreditato al saldo dell'utente.		

Nome	Interfaccia	Descrizione
utilizzaSaldo	SaldoService	Scala un importo dal credito dell'utente.
Precondizioni		
L'importo da scalare deve essere positivo. Il saldo attuale dell'utente deve essere maggiore di zero.		
Postcondizioni		
Il nuovo saldo dell'utente è pari al saldo precedente meno l'importo. Viene registrata l'operazione.		

Nome	Interfaccia	Descrizione
validaBiglietto	ValidazioneService	Effettua una verifica della validità di un biglietto tramite QR Code all'ingresso della sala.
Precondizioni		
Deve esistere un biglietto nel sistema con quel codice QR. Il biglietto deve essere nello stato *EMESSO*. La data della programmazione correlata al biglietto deve coincidere con la data di oggi.		
Postcondizioni		
Lo stato del biglietto viene aggiornato con *VALIDATO*. Viene registrata la validazione.		

7.1 Invarianti delle classi

Di seguito sono riportate le regole che devono essere sempre vere per le entità del sistema.

Nome classe: Sala
Invarianti
La capienza della sala deve essere sempre uguale al prodotto di numeroFile per numeroPostiPerFila. Una sala non può avere capienza negativa o nulla.

Nome classe: Programmazione
Invarianti
Una programmazione non può terminare prima di iniziare quindi l'ora di fine deve essere successiva all'ora di inizio. Una programmazione non può avere un prezzo base negativo.

Nome classe: Biglietto
Invarianti
Un biglietto non può esistere senza collegamenti a un Acquisto, un posto e a una Programmazione. Il prezzo finale del biglietto non può essere negativo.

Nome classe: Posto
Invarianti
Un posto deve appartenere univocamente a una sola Sala.

8. Design patterns

Durante la definizione dei servizi del sistema Tickema, ci si è resi conto di come la progettazione basata su un'architettura a 3 livelli (Presentation, Service, Repository) presentasse criticità strutturali che richiedevano soluzioni di disaccoppiamento.

8.1 Facade Pattern

La prima criticità riguarda la gestione del flusso di acquisto biglietti (UC_UA_2).

Il processo di acquisto biglietti coinvolge molteplici servizi del sistema che devono collaborare in modo coordinato:

1. ProgrammazioneService
2. PostoService
3. TariffaService
4. AcquistoService
5. PagamentoService
6. SaldoService
7. BigliettoService

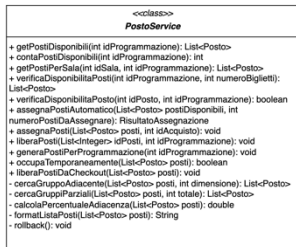
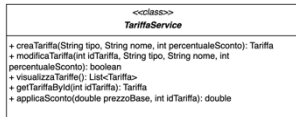
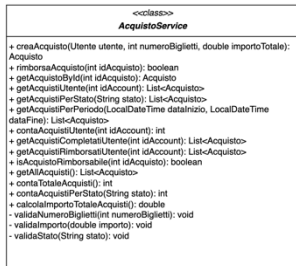
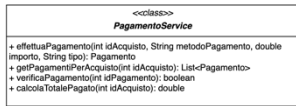
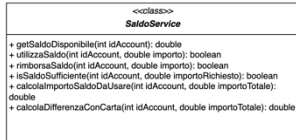
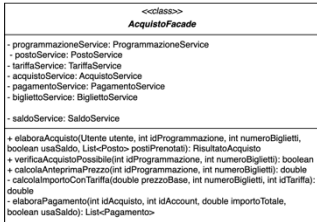
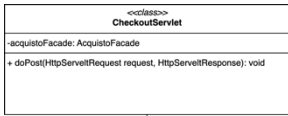
È necessario, infatti, gestire la verifica della disponibilità dei posti, il calcolo dei prezzi applicando le tariffe, l'assegnazione automatica dei posti con adiacenza quando possibile, l'eventuale elaborazione della transazione economica e l'emissione dei biglietti digitali.

Senza un livello di coordinamento, il Controller dovrebbe:

- Conoscere e istanziare tutti questi servizi (alto accoppiamento)
- Orchestrare manualmente la sequenza di chiamate (logica business nel Presentation layer)
- Gestire transazioni e rollback in caso di errore (complessità eccessiva)
- Esporre dettagli implementativi che dovrebbero rimanere nascosti

Il Facade Pattern risolve questo problema introducendo la classe **AcquistoFacade** nel livello Service. Questa classe agisce come punto di accesso unificato, esponendo un'interfaccia semplificata al Controller e nascondendo la complessità interna.

Questa classe coordina internamente le chiamate ai vari sottosistemi seguendo una sequenza ben definita.



Lo schema è consultabile in alta risoluzione scaricando il relativo file PDF dalla cartella; la visualizzazione diretta tramite anteprima Drive potrebbe comprometterne la nitidezza.

8.2 Strategy pattern

La seconda criticità riguarda la gestione dei metodi di pagamento nel sistema. Il requisito RF-UA_11 specifica che gli utenti devono poter pagare utilizzando tre modalità diverse: carta di credito, saldo disponibile o pagamento misto (combinazione di saldo e carta), ciascuno con logica specifica.

1. Pagamento con Carta di Credito

- Validazione dati carta (numero, scadenza, CVV)
- Esecuzione pagamento
- Generazione codice autorizzazione
- Gestione errori di pagamento rifiutato

2. Pagamento con Saldo

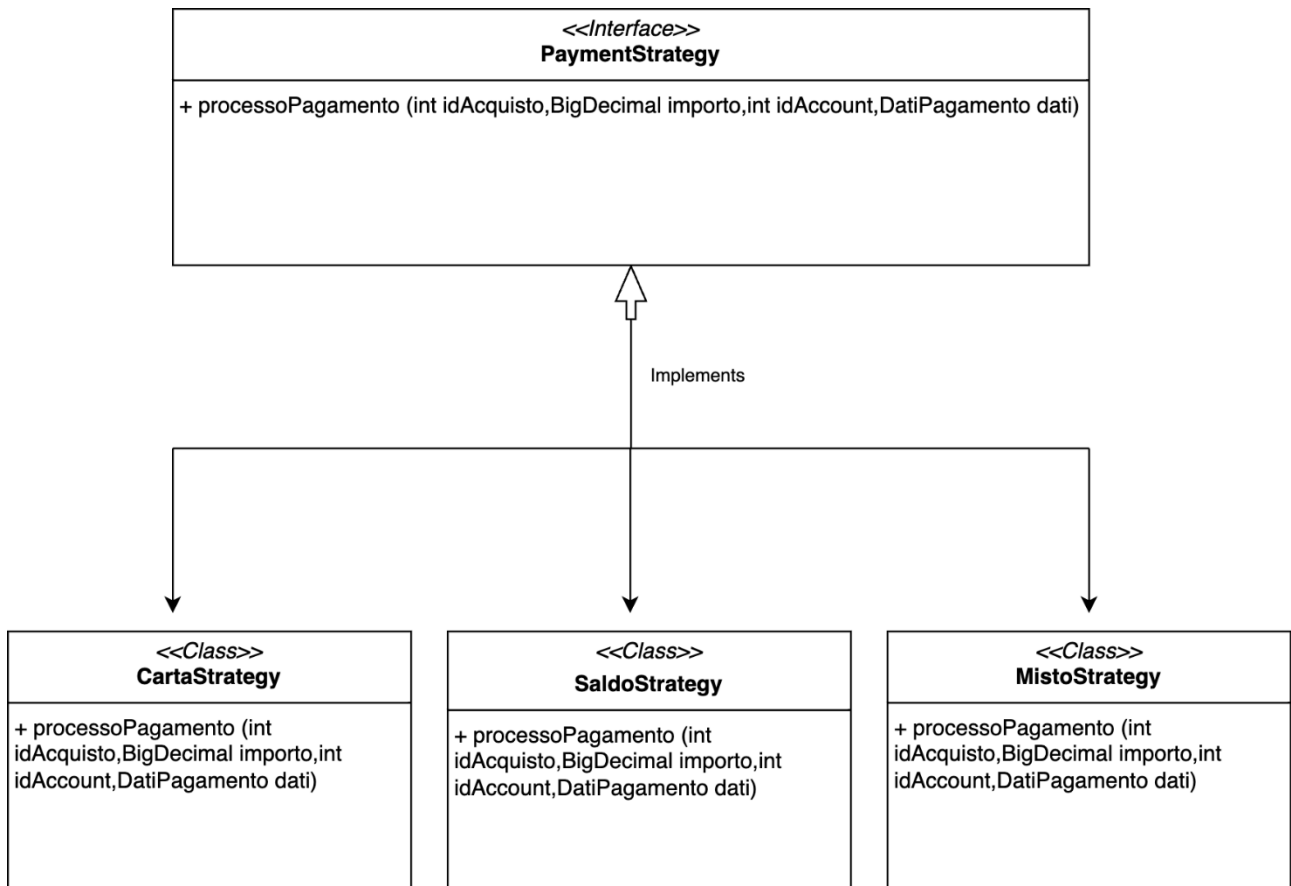
- Verifica disponibilità saldo sufficiente
- Decremento saldo dell'account utente
- Registrazione movimento
- Gestione eccezione saldo insufficiente

3. Pagamento Misto (Saldo + Carta)

- Utilizzo totale del saldo disponibile
- Calcolo differenza rimanente
- Processamento differenza con carta
- Rollback saldo in caso di fallimento carta
- Creazione di due record Pagamento distinti (ACCONTO e SALDO)

Implementare questa variabilità con strutture condizionali tradizionali (if-else) avrebbe portato a codice rigido e difficilmente estensibile. Inoltre, l'aggiunta di eventuali nuovi metodi di pagamento futuri (come PayPal o bonifico) avrebbe richiesto modifiche invasive al codice esistente.

Lo Strategy Pattern risolve questo problema definendo una famiglia di algoritmi intercambiabili. Ogni metodo di pagamento diventa una classe concreta che implementa l'interfaccia PaymentStrategy.



Lo schema è consultabile in alta risoluzione scaricando il relativo file PDF dalla cartella; la visualizzazione diretta tramite anteprima Drive potrebbe comprometterne la nitidezza.